

Week 9 Lab: AstraNotes Test Improvement Log

Student: Alex Shirazi | Project: AstraNotes |

1. Feature and Requirement Reviewed

Feature reviewed: keyword search (FR-07), specifically the case-insensitive match across note title and body. The implementation lives in `NoteService.search()` and the tests are in `tests/test_search.py`, added during the Week 8.1 branch refactor.

Secondary feature reviewed: edit with invalid title. `NoteService.edit()` calls `ValidationLayer.validate_title()` before writing — the same rule enforced on create — but no test exercises this path.

2. Weak Test Identified

File: tests/test_search.py, function: test_search_matches_title_case_insensitive

Original test:

```
def test_search_matches_title_case_insensitive(populated_service):
    result = populated_service.search("SPRINT")
    assert result.count >= 1
```

The populated fixture contains three notes: "Sprint Zero Notes" (title match), "API Key" (no match), and "Meeting Notes" with body "Sprint review on Friday" (body match). Searching "SPRINT" should return exactly 2 notes. The assertion `result.count >= 1` passes even if the implementation is broken and returns only 1 result. It would also pass if search accidentally returned all 3. This is a noisy assertion — it gives a green signal without actually verifying the correct behavior.

3. Improved Test

```
def test_search_matches_title_and_body_case_insensitive(populated_service):
    """
    FR-07: case-insensitive search returns exactly the notes
    whose title or body contains the keyword.
    'SPRINT' matches:
    - 'Sprint Zero Notes' (title match)
    - 'Meeting Notes'      (body: 'Sprint review on Friday')
    It does NOT match 'API Key' / 'Store in environment variable'.
    """
    result = populated_service.search("SPRINT")

    # Exact count — not >= 1
```

```

assert result.count == 2

# Verify the right notes came back, not just any two
returned_titles = {n.title for n in result.notes}
assert "Sprint Zero Notes" in returned_titles
assert "Meeting Notes" in returned_titles
assert "API Key" not in returned_titles

```

The improved test pins the exact expected count and asserts on the identity of the returned notes. A regression that drops one match or returns an extra note will now fail immediately with a clear message.

4. What Was Wrong With the Original

The original assertion (`>= 1`) is what the assignment calls a weak assertion: it confirms that search returned *something* but not that it returned the *right* things or the right number of things. This pattern is common when tests are written quickly to hit coverage targets. The test would have passed even if the search implementation had a bug that only matched titles and completely ignored bodies — because the title-only match on "Sprint Zero Notes" is enough to satisfy `>= 1`. The test was green when it should have been catching a real gap.

5. Mocking: Helping or Hiding Risk

The current tests use no mocking. The service fixture wires a real `JsonFileRepository` to `pytest's tmp_path`, so every test exercises the actual file I/O path. This is the right call for this project.

The honest tradeoff: these tests are really integration tests dressed as unit tests. If disk I/O were slow or a file permission issue caused a failure, the test output would blame `search` or `create` rather than storage. For a 500-note local app with no network dependencies, the risk of `tmp_path` flakiness is low and the value of testing the real persistence path is high. Mocking the repository here would have hidden whether `list_all()` actually returns the notes that were saved — which is exactly what the tests need to verify.

Verdict: no mocking is the correct decision. The tests are honest about what they're exercising.

6. Meaningful Coverage Gap That Still Matters

Gap 1 — `edit()` with invalid title: `NoteService.edit()` calls `ValidationLayer.validate_title()` before writing, but no test exercises this path. The coverage report would show the `edit()` method as covered because the sort test calls it — but the validation branch inside `edit` is never triggered. A test like `test_edit_empty_title_raises_validation_error` is missing.

Gap 2 — corrupt file recovery in `list_all()`: `JsonFileRepository.list_all()` silently skips corrupt JSON files and continues loading the rest. This behavior is exactly right per the Week 3.1 edge

case review, but it is completely untested. A coverage tool would show the except branch as uncovered. A test that writes a malformed .json file to tmp_path and then calls `service.list_sorted()` would verify that the app doesn't crash and still returns the valid notes.

Why coverage doesn't tell you this: Both gaps involve branches that are reachable but never triggered by the current test suite. Line coverage will show those methods as covered because the happy-path lines run. Branch coverage would expose them, but branch coverage is not currently configured in the test setup.

7. How AI Helped Critique the Test

AI was used to review `test_search.py` after writing it. The critique pass identified the `>= 1` assertion as a weak signal and suggested replacing it with an exact count. AI also flagged that the test did not verify which specific notes were returned, only how many — which is the same weakness from a different angle.

AI additionally pointed out that `test_search_keyword_stripped_before_match` and `test_search_result_contains_keyword` both test the same stripping behavior and could be merged into one test without losing signal.

AI did not identify the `edit()` validation gap or the corrupt file gap — both were found by manually tracing the call graph from `NoteService` through `ValidationLayer` and `JsonFileRepository`.

8. What Was Accepted, Changed, or Rejected

Accepted: Replace `>= 1` with exact count `== 2` in the case-insensitive search test. The reasoning was correct and the fix is straightforward.

Accepted: Add title identity assertions to confirm which specific notes were returned, not just the count.

Rejected: AI suggested merging the two stripping tests into one. Rejected — they test slightly different things (the keyword field vs. the match count) and keeping them separate makes the failure message more informative.

Rejected: AI suggested mocking `JsonFileRepository` to isolate `NoteService` as a pure unit test. Rejected — the real repository with `tmp_path` provides more honest coverage at this scale and the isolation benefit does not outweigh the loss of real I/O verification.

Added manually: Identified the `edit()` validation gap and corrupt file recovery gap through manual code tracing. AI did not flag either of these.